

Production Deployment

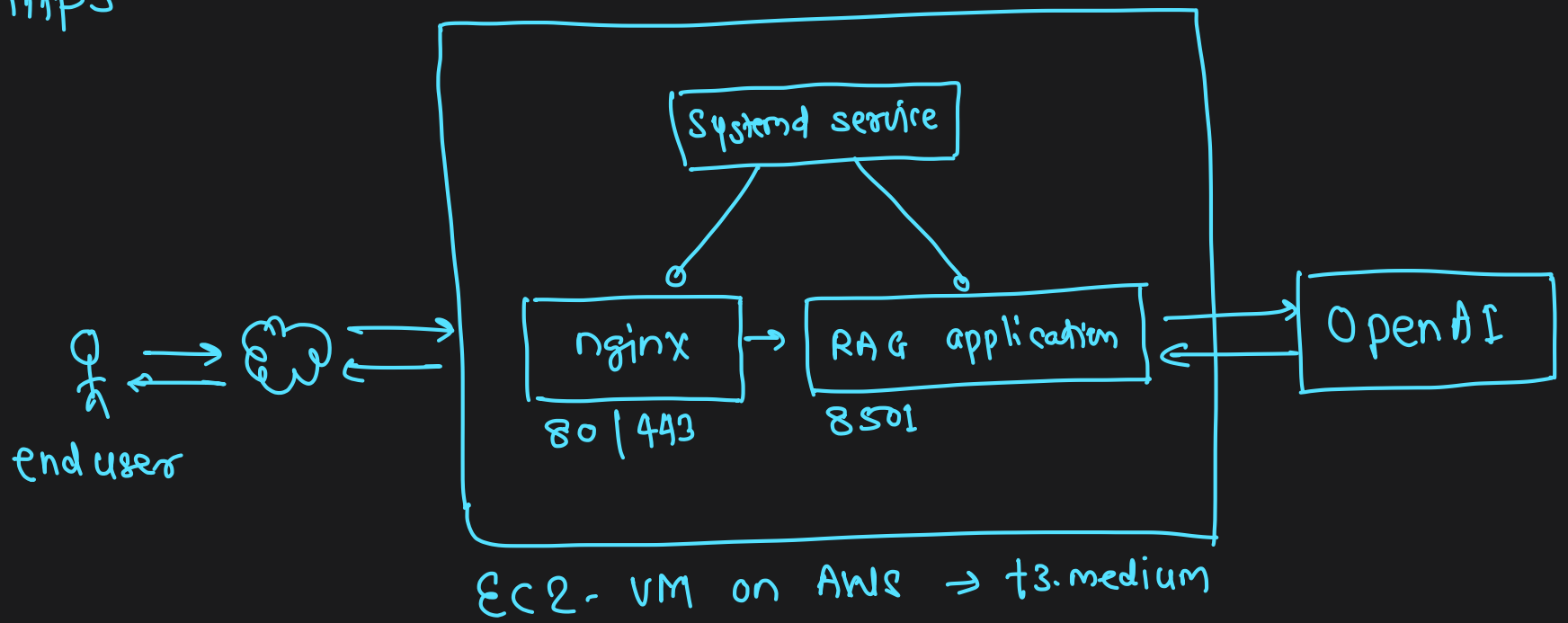
making the application available to end users

Architecture Overview

① nginx → frontend deployment

② cloud → AWS

③ http | https



Cost Considerations

- **EC2 instance costs (approximate):**
  - t3.small: ~$0.02/hour (~$15/month)
  - t3.medium: ~$0.04/hour (~$30/month)
- **API costs dominate:**
  - OpenAI embedding: $0.02/1M tokens
  - GPT-4o-mini: $0.15/1M input, $0.60/1M output
- **Managed alternative comparison:**
  - Bedrock Managed Knowledge Base: ~$350/month fixed floor (OpenSearch Serverless 2 OCU minimum)
  - Self-managed on EC2: Near-zero at low volume, scales with usage
- **Decision rule:** EC2 self-hosting wins at low-to-medium volume or when you need full control

Instance Selection

- Development / Test
  - Type: t3.small (2 vCPUs / 2 GiB)
  - Minimum viable
- Production CPU
  - Type: t3.medium or c7g.xlarge (2-4 vCPU / 4-8 GiB)
  - 4 GiB RAM recommended
- GPU Inference
  - g4dn.xlarge: (4 vCPUs/ 16 GiB + T4)
  - For local model serving
- Storage: 20 GB EBS minimum
- Security Groups:

Firewall

Port	Protocol	Source	Purpose
22	TCP	Your IP only	SSH access
80	TCP	0.0.0.0/0	HTTP (redirects to HTTPS)
443	TCP	0.0.0.0/0	HTTPS traffic

Systemd Service



```
[Unit]
Description=Agentic RAG Service
After=network.target

[Service]
User=ubuntu
Group=ubuntu
WorkingDirectory=/home/ubuntu/rag
ExecStart=/home/ubuntu/.local/bin/streamlit run app.py --server.address=0.0.0.0
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Nginx Reverse Proxy



```
map $http_upgrade $connection_upgrade {
    default upgrade;
    ''      close;
}

server {
    listen 80;

    location / {
        proxy_pass http://127.0.0.1:8501;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_set_header Host $host;
        proxy_read_timeout 86400;
    }
}
```



Optimizing Production Environment

Optimization Triangle



- **Efficiency**
  - Right-sized resources, no over-provisioning
  - Prevents resource contention that cascades across workloads
- **Resilience**
  - Components fail independently
  - Limits failure propagation to one component
- **Operability**
  - Automated, auditable management
  - Eliminates human error as a failure vector

Blast Radius for Agentic RAG

- **Definition:** The scope of resources, data, and capabilities affected if a component fails or is compromised
- For an Agentic RAG system, blast radius components include

Component	Risk Factor	Blast Radius Amplifier
LLM API credentials	Compromised key = unlimited spend	High
Vector store	Corruption = all retrieval fails	High
Agent orchestration	Infinite loop = resource exhaustion	Medium
EC2 instance	Single point of failure = total outage	Critical
Security group	Overly permissive = lateral movement	Critical

Instance Right-Sizing (Efficiency + Blast Radius)



- **AWS right-sizing guidance:** Identify instances with **maximum CPU and memory utilization below 40%** over 4 weeks — these are candidates for downsizing
- **Monitoring approach:**
  - Focus on recent data (not historical)
  - Only evaluate instances running at least half the time
  - Ignore T-series burst instances for CPU-based sizing decisions
- **Blast radius benefit:**
  - A right-sized instance has predictable failure modes
  - An over-provisioned instance creates the illusion of capacity, hiding bottlenecks until they cascade

Security Group Hygiene (Blast Radius Containment)

- **AWS best practice:** Apply **least-privilege rules** to security groups .
- **Anti-pattern (high blast radius):**
  - Inbound: 0.0.0.0/0 → Port 22 (SSH)
  - Inbound: 0.0.0.0/0 → Port 8000 (FastAPI direct)
- **Critical rule:**
  - EC2 Security Group and on-host firewall (UFW/iptables) are **separate layers**
  - Both must enforce least privilege
- **Blast radius impact:**
  - Removing SSH eliminates the most common lateral movement vector
  - An attacker cannot pivot to the instance even if other credentials leak

Production Checklist



- **Efficiency:**
  - Instances right-sized based on 4-week max CPU/memory < 40%
  - Spot instances used only for interruption-tolerant workloads
  - Auto Scaling for stateless components
- **Blast Radius Containment:**
  - Security groups follow least-privilege (no 0.0.0.0/0 on SSH)
  - SSH disabled; Systems Manager used for access
  - Separate credentials per service/component
  - Vector store not publicly accessible
  - Multi-AZ for stateful components
- **Resilience:**
  - Automated health checks and remediation
  - Gradual deployment with canary stages
  - Agent loop iteration limits enforced
  - API spend limits configured
- **Observability:**
  - All layers monitored (infrastructure → agent behavior)
  - Alerts tuned for early detection, not noise
  - Change dependency graph traced before modifications

Evaluation

RAGAS



- RAGAS (Retrieval Augmented Generation Assessment) is an evaluation framework for LLM applications that uses LLM-as-a-judge to score quality without requiring massive human-annotated datasets
- **The problem it solves**
  - Traditional metrics like BLEU or ROUGE fail to capture what matters in RAG systems—factual consistency, retrieval quality, and answer relevance
  - RAGAS provides **quantifiable scores** for each failure point in the RAG pipeline
- **Why it matters for production**
  - Without systematic evaluation, you ship blind
  - RAGAS turns "answers feel better" into measurable deltas—faithfulness up, retrieval stable, cost per query down

Core RAGAS Metrics

Metric	What It Evaluates	Failure Mode Detected
Faithfulness	Is the answer grounded in retrieved context?	Hallucination—model freelancing
Answer Relevancy	Does the answer address the query?	Off-topic or incomplete responses
Context Precision	Are relevant documents ranked higher?	Poor retrieval ranking
Context Recall	Does context contain all needed information?	Missing key facts from retrieval

Recommended production targets : Faithfulness ≥ 0.9, Answer Relevancy ≥ 0.85, Context Precision ≥ 0.8